

# scripts/autonomous-qa/run-matrix.sh — User Guide

**Last verified:** 2026-07-26 (LVA-018 script-docs backfill cycle)  
**Inheritance:** HelixConstitution §11.4.18 (script docs), §11.4.128 (evidence layout); Lava §6.AG / §6.AH / §6.X (containerized emulators), §6.J (anti-bluff verdicts) **Classification:** project-specific

## Overview

Orchestrates the autonomous-QA backend × provider × query matrix — **ONE backend at a time** (operator rule: never run both APIs at once):

backend up → boot ONE containerized KVM emulator (kept alive)  
→ for each provider-subset × query:  
    FRESH install + record + run Challenge70 (via run-iteration.sh)  
→ tear emulator down → backend down → write curated summary.md

Plan: docs/superpowers/plans/2026-06-29-autonomous-qa-backend-provider-matrix.md.

## Usage

```
scripts/autonomous-qa/run-matrix.sh --backend goapi|apiapp \  
  [--queries 1080p,mp3] [--subsets all] [--external-backend]
```

```
# Keystone (Phase 0): single provider, single query  
scripts/autonomous-qa/run-matrix.sh --backend goapi --subsets  
  rutracker --queries 1080p
```

| Flag                        | Meaning                                                                        |
|-----------------------------|--------------------------------------------------------------------------------|
| --backend goapi apiapp      | Which API backend the on-device client targets (required)                      |
| --queries <csv>             | Search queries to run per subset (default 1080p,mp3)                           |
| --subsets all <csv1>;<csv2> | all = every non-empty subset of the API-backed providers (31 subsets, see lib- |

| Flag               | Meaning                                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                    | subsets.sh); otherwise ;-separated CSV groups, e.g. "rutracker;kinozal,nmclub"                                                         |
| --external-backend | Assume the backend is already up + healthy out-of-band (e.g. thinker via distribute-api-remote.sh); the script skips bring-up/teardown |

Only **API-backed** provider ids (QA\_PROVIDERS in lib-subsets.sh: rutracker, nmclub, kinozal, archiveorg, gutenbergrutor (bundled-only) and iptorrents (Jackett-only) are dropped with a warning — neither API backend vends them via GET /v1/providers, so Challenge70 would only SKIP opaquely inside the emulator.

## Evidence layout

```
.lava-ci-evidence/autonomous-qa/<date>/<backend>/
  summary.md                # curated matrix table + totals
  <subset-slug>-<query>/    # per-iteration evidence (see run-
iteration.sh)
  verdict.json  junit.xml  raw/
```

## Anti-bluff properties (§6.J)

- Any stale verdict.json is deleted **before** each iteration, so a prior-run PASS can never be misread when run-iteration.sh dies early (sixth-law-incidents 2026-07-03 missing-APK/stale-verdict PASS bluff).
- An iteration with no verdict.json is recorded **FAIL** (a setup error is never a pass — no test executed).
- A non-zero iteration exit code can **never** be counted PASS, even if the verdict file says PASS (defense-in-depth).
- Unknown/garbled verdict tokens classify as FAIL (qa\_classify).

## Exit codes

- 0 — matrix ran to completion (per-iteration verdicts live in summary.md; the script itself does not fail on iteration FAILs)
- 2 — usage error, unknown backend, or no API-backed subsets remain after filtering

## Companion files

- scripts/autonomous-qa/run-iteration.sh — one matrix cell (install, record, run, verdict)

- `scripts/autonomous-qa/lib-subsets.sh` — provider-subset enumeration
- `scripts/autonomous-qa/lib-emulator.sh` — containerized KVM emulator lifecycle
- `scripts/autonomous-qa/lib-backend.sh` — backend bring-up / targeting / teardown
- `scripts/autonomous-qa/lib-summary.sh` — verdict parsing / classification helpers
- `scripts/autonomous-qa/aggregate-evidence.sh` — rolls the matrix up into the §6.AK distribute-gate artifacts